Research status and transfer

Prepared for EJ | 22 October 2026

No Rule 30 Prize conjecture has been proved or disproved. No native Effortless build completed. The useful output is an audited research package, elementary partial arguments, finite computational results, recorded falsifications, and an unbuilt native rulebook draft.

What was completed

The v05 research record documents inspection of all 15 files in the supplied v04 archive, reruns of all four original Python experiments, independent sequence checks, new mathematical derivations and multiple hypothesis/revision cycles. Its reports, code, numerical outputs and original inputs are preserved in the companion ZIP.

Official Rule 30 specifications, foundational CMCC and BRCC papers, and Ruliad material were consulted. The Simpson's Paradox and Effortless Math/Fermat examples were inspected for their correction and trust workflows; their entire mathematical content was not independently certified.

Measure	Actual state
Research iterations	13 legacy cycles: report cycles 0-11 plus one follow-up
Native Effortless loops	0 verified
Executed SQLite database	6 tables; 3 views
Native rulebook draft	8 proposed tables; formulas not executed
Claims and revisions	26 claims, 18 direct dependency links, 5 revisions
Proof assistants / Wolfram	Neither Lean nor Wolfram Language executed

Critical correction to earlier framing

The implemented CMCC-style model used custom Python and SQLite. It was not an Effortless rulebook build. Its database views and ledger do not demonstrate that every research loop consumed a rebuilt, complete inference state. The user made native Effortless builds an essential requirement; this session did not satisfy it.

Files to take to the next environment

cmcc_rule30_effortless_migration.zip contains both **research_v05/** and **effortless_rule30/**. Start with research_v05/REPORT.md for the complete partial proofs and effortless_rule30/README.md for the migration limits. This PDF adds the subsequent source-checkout diagnosis, which postdates that ZIP.

This handoff summarizes retained records; it is not a fresh external peer review of the proofs. Novelty is not claimed, and no prize submission was made.

Strongest mathematical results

Model: x(i,0)=1 exactly when i=0, with an infinite zero background. For t >= 0:

```
x(i,t+1) = x(i-1,t) XOR (x(i,t) OR x(i+1,t))
c(t) = x(0,t)
y_k(t) = x(t-k,t), y_-1(t)=0
y_k(t+1) XOR y_k(t) = y_(k-1)(t) OR y_(k-2)(t)
```

A. Exact period from forcing parity

If q has least positive period Q and y(t+1)=y(t) XOR q(t), let e be the XOR of q over one Q-cycle. The least period of y is Q when e=0 and 2Q when e=1. Summing over Q steps gives y(t+Q)=y(t) XOR e. Every period of y is also a period of q and hence a multiple of Q; this establishes minimality. This yields a full-cycle exact-period algorithm for the fixed diagonals.

B. Unbounded fixed-diagonal periods

For k >= 1, the first 1 in y_k occurs at ceil(k/2). Before then its position lies outside the cone; at that time it is on the left boundary or its first interior cell, both equal to 1. A period at most ceil(k/2) would move that first 1 to an earlier time. Thus P_k > ceil(k/2), while the original induction gives P_k dividing 2^k.

C. Explicit adjacent-column mismatch window

For every T >= 0 and p >= 1, some t in [T,2T+p] and i in {0,1} satisfy x(i,t) != x(i,t+p). Inverting the rule gives x(i-1,t)=x(i,t+1) XOR (x(i,t) OR x(i+1,t)). If both columns agreed with their p-shifts throughout that window, agreement would propagate left to (-T-p,T). But that cell is 0 at T and 1 at T+p by cone support and the boundary invariant. Contradiction.

D. Constant center-run bounds

If c is constantly 1 on [T,V], then V <= 2T+1. If it is constantly 0, then V <= 4T+5. For a 1-run the left neighbor is forced to 0; for a 0-run the right neighbor is nondecreasing and can switch at most once. Apply the adjacent-column window obstruction to the resulting constant segments. Full endpoint details are in REPORT.md, section 5.

These bounds exclude eventual constant tails and imply that both colors occur infinitely often. The qualitative period-one exclusion is a **rederived known result**, not a new prize solution. None of these results excludes all nonconstant periodic center tails.

E. Input sensitivity and representation equivalence

The n-step center output for arbitrary input bits equals its extreme-left input XOR a function of the remaining inputs. Induction proves input-ensemble balance, but not temporal balance for the single seed. Separately, induction in time proves that correctly indexed CMCC cell equations uniquely reproduce Rule 30. Neither result establishes a fixed-seed complexity lower bound.

Experiments, falsifications and gaps

Computation	Retained result
Center through t=100,000	100,001 bits: 50,098 ones, 49,903 zeros; two implementations agree
Finite period exclusions	All p=1..10,000 excluded for every onset T <= 90,000
Exact diagonal periods	Certified through k=51; P_51=2,097,152
CMCC-style cell model	33,411 SQLite cells through t=128 agree with independent evolution
Adjacent windows	6,464 tested (T,p) windows have witnesses
Forcing words	All 1,966 primitive words of length <=10 checked
Relaxed neighbor probe	Every tested nonconstant primitive word permits recurrent branching

The finite results do not establish an asymptotic law. The diagonal computation stopped at a preset allocation cap, not at a proved mathematical barrier. The neighbor probe treats the next column as free input and therefore does not enforce the full lattice or seed.

Failed hypotheses retained

All nonboundary diagonals are balanced: false; y_5 has least-period word 00010101, with 3 ones out of 8. **Periods always double:** false; k=4 and k=5 both have period 8. Replace both shortcuts with the forcing-period/parity theorem.

Unbounded diagonal periods force center aperiodicity: unsupported. An explicit auxiliary array has the same cone support, first-arrival times and unbounded fixed-diagonal periods, but an eventually zero moving diagonal. It violates Rule 30, demonstrating exactly why the recurrence coupling cannot be omitted.

ACID implies order independence: false without additional hypotheses. Starting at 0, add-one then double gives 2, while the reverse order gives 1. **Arbitrary aggregation nesting preserves answers:** false; the mean of the means of [0] and [10,10,10] is 5, whereas the pooled mean is 7.5. Preserve sums and counts.

Unresolved prize obligations

Problem 1: force a center mismatch for every candidate period p >= 2 and every onset. The adjacent-column theorem allows all witnesses to occur in the neighbor. Seed-compatible constraints across additional columns are still missing.

Problem 2: prove a sublinear discrepancy bound for |2 sum(t<N)c(t)-N|, or another sufficient convergence argument. Balance over initial states and finite frequencies do not supply it.

Problem 3: fix the computation model, encoding, costs and exact quantifiers before attempting a lower bound over all algorithms. The report flags a discrepancy between prose and a displayed predicate in the announcement; recheck the official wording before any submission. Large-integer updates are not unit-cost bit operations.

Native Effortless migration status

This is a schema draft, not a successfully built native database. The original SQLite database remains the only executed database in this package.

Draft table	Rows	Purpose
Assertions	26	Statements, imported current statuses and proof references
Dependencies	18	Original direct premise/conclusion edges
Prerequisites	58	Self and transitive ancestors; premise lookups
Assessments	26	SUMIFS rollups and calculated blocking status
Evidence	8	Artifact paths and SHA-256 values
Revisions	5	Prior/new status, reason and replacement claim
Loops	13	Legacy research cycles only
Dashboards	1	Aggregate claim and native-loop counts

Intended formula behavior

Prerequisites look up premise usability and proof eligibility. Assessments aggregate blocked prerequisites and calculate the current status. Finite experimental verification can count as usable evidence without counting as proof. An ancestor becoming unusable should block dependent assessments after a real build. Original reported statuses remain visible so open and falsified assertions are not conflated.

What local validation established

validate_draft.py checks unique identifiers and field names, foreign-key targets, an acyclic table relationship graph, an acyclic claim graph, and exact transitive prerequisite closure. It passed. It does not validate official transpiler compatibility, execute formulas, generate SQL, or query generated views.

Remaining completeness gaps

The original ledger lacked evidence-to-claim mappings; the draft does not invent them. Revision records are retained but are not an executed revision-application engine. The finite cell model has not been migrated into native Effortless formulas. Adding dependency edges requires closure regeneration and validation. Human-readable proof statuses are imported assertions, not machine-checked proof certificates.

migrate.py is a one-time importer. Rerunning it overwrites the draft from the archived v05 source. Do not run it after editing the native hub unless that overwrite is intentional.

Required propagation test

In a disposable fixture, build the baseline, change T0 to FALSIFIED, rebuild and query the generated assessment views. TB, TC, TD and K1 must become blocked through their dependencies. Restore the baseline and rebuild. Preserve input hashes, build receipts and query results; only a completed, verified native cycle may increment the Effortless loop count.

CLI attempts and environment blocker

Official repository: github.com/EffortlessAPI/cli. Installed npm package: @effortlessapi/cli, version 2026.1008.1747, with its matching linux-x64 platform package.

Published binary attempt

The platform package contained a precompiled, executable Linux x86-64 binary of about 36 MB. Running it through the Node launcher and directly both failed before application startup:

```
Failed to create CoreCLR, HRESULT: 0x8007000E
Exit code: 137
```

Changing the bundle extraction directory, imposing smaller GC heap limits, reducing processor count, disabling diagnostics, and changing WriteXorExecute did not resolve startup. These were diagnostic attempts, not successful builds. Exit 137 alone is not proof of an operating-system out-of-memory kill.

Source checkout attempt

The repository was cloned locally at commit:

```
3e79aaba23dd925ed3c184155bfbc7235137b84f
```

npm dependencies installed. Running npm start -- -version selected the source-checkout developer path and returned:

```
Effortless CLI: developer mode (building from source in this checkout)...
/bin/sh: 1: dotnet: not found
```

This source path has a different requirement from the published binary. The current main branch targets net8.0 and bundles its runtime for publication. The separate .NET installation requirement is removed for published-package users; the application still uses .NET internally. The checkout is not a complete Node implementation.

What is known and unknown

Host tracing reached bundled runtime initialization. /proc/self/maps, /proc/self/status and /proc/meminfo were unavailable; syscall tracing was denied. Basic memory reporting and small executable-memory mapping tests succeeded. An environment-specific startup incompatibility is plausible, but the precise failing operation was not established. This is not evidence that the CLI fails on the user's or team's working machines.

The documented remote MCP endpoint answered health and tools/list, but exposed only ping/build/query/validate. Its build tool needs a server-local project path; the local project was not transferred or built there. No authentication request, remote publication, or external repository modification was performed.

Practical continuation

Use the user's known-working Effortless environment. First verify effortless -version there, then initialize and build the native project with official tools. Do not spend additional research time reproducing this session's runtime restrictions unless diagnosing this environment is itself a goal.

Reproduction and continuation checklist

Companion archive navigation

```
cmcc_rule30_effortless_migration.zip
research_v05/REPORT.md Complete partial proofs and research narrative
research_v05/SOURCES.md Sources and exact retrieval scope
research_v05/README.md Reproduction commands and trust boundary
research_v05/input/ Untouched supplied research
research_v05/rerun/ Original scripts and regenerated results
research_v05/data/ Sequences, witnesses, JSON and SQLite
research_v05/sources/ Foundational PDFs and extracted text
effortless_rule30/ Native draft, importer and failure receipts
```

Python reproduction

Python 3.10+ standard library and its SQLite module suffice. From research_v05/, run the following. Scripts regenerate metadata and may change timing/environment hashes; scientific counts and witnesses should agree. Read README.md before comparing the original MANIFEST.json after regeneration.

```
python rerun/experiment.py
python rerun/experiment_v02.py
python rerun/experiment_v03.py
python rerun/experiment_v04.py
python research.py
python structural.py
python adversarial.py
python verify_periods.py
python periodic_center_probe.py
python ledger.py
python verify_bundle.py
```

Start the next native research loop

1. Read REPORT.md and the migration README. Independently check the partial proofs; do not treat ledger statuses as proof certificates.
2. On a working CLI host, initialize the project and install the official minimize-rulebook and rulebook-to-postgres pipeline steps. Validate and correct schema compatibility.
3. Run effortless build, execute generated PostgreSQL outputs and consume generated views. Preserve raw build receipts.
4. Run the controlled dependency mutation test from page 4. Add evidence mappings and revision handling before claiming a complete inference picture.
5. Record each subsequent hypothesis, experiment, counterexample and revision in the hub. Rebuild and inspect the current views before choosing the next hypothesis.

Research priority

The most concrete next mathematical attack is Problem 1: constrain a proposed nonconstant periodic center using additional right-hand columns and the single seed. The relaxed one-neighbor graph already failed as a universal obstruction. A finite-width search must explicitly preserve or quantify its boundary inputs; arbitrary zero or periodic boundaries would change the problem.

Do not report success from numerical regularity, a clean database build, a renamed proof obligation, or a finite graph relaxation. A prize claim requires a complete argument for the original infinite statement.

Sources, corrections and transfer notes

Primary mathematical sources

Rule 30 Prize specifications
https://rule30prize.org/

Wolfram: Announcing the Rule 30 Prizes (2019)
https://writings.stephenwolfram.com/2019/10/announcing-the-rule-30-prizes/

WolframInstitute Rule30PrizeKit
https://github.com/WolframInstitute/Rule30PrizeKit

CMCC foundational paper
https://doi.org/10.5281/zenodo.14760293

BRCC proof sketch
https://doi.org/10.5281/zenodo.14759299

Wolfram: The Concept of the Ruliad
https://writings.stephenwolfram.com/2021/11/the-concept-of-the-ruliad/

Effortless sources

Skills and native build workflow
https://github.com/EffortlessAPI/effortless-skills

CLI source, packaging and README
https://github.com/EffortlessAPI/cli

Simpson and Effortless Math examples
https://github.com/EffortlessAPI/effortless-rulebooks

The research examples were inspected at commit afe180c67cd8b0247c9b82ec72aa87e3654efd9a. See SOURCES.md for exact files, the limits of review, and foundational-source revisions.

Corrections the next researcher must retain

Stored sequence hashes use digit strings without a trailing newline. The audit also records full-file hashes. Inclusive horizon T gives T+1 bits; at T=10,000 there are 5,033 ones in 10,001 bits, whereas the first 10,000 bits contain 5,032 ones. Finite observed periods and exact full-cycle periods are distinct. The diagonal center identity is c(t)=y_t(t), not a fixed-offset sequence.

The report's claim that a CMCC representation was implemented refers to custom SQLite. It must not be read as a claim of a native Effortless build. Likewise, references to claim/evidence links describe an intended model more broadly than the actual ledger: explicit evidence-to-claim mappings remain incomplete.

Transfer integrity

Companion archive SHA-256:

`19f49881e1fcd9aacbaabd93472cbe15593ec67532618de82e25868924ec09b9`

The ZIP predates the latest source-clone test. The checkout commit and exact failure are therefore recorded in this PDF. The archive retains the original research plus the migration draft; no successful build receipt exists.